Resit Exam prep [Python]

Mentor Program

2024 - 2025

WhatsApp Community

Website

Instagram

minigames available

Stay Connected

We are
NOT TAs

Disclaimer!

- print(“is your best friend”)
- Learn to read error messages
- Make a battle plan before you start coding (break it down into parts)
- Get fast & make mistakes
- **LBYL**: Look before you leap
- **EAFP**: It's easier to ask for forgiveness than permission

General tips!

Study plan & resources

- W3Schools Python Tutorial
- GeeksforGeeks
- Slides & Exercises
- Cheatsheet(s)

- Advanced:
  - Codewars
  - Leetcode

- Friday:
  - Go over basics
- Saturday:
  - Dictionaries
  - Nesting
- Sunday:
  - OOP
- Monday:
  - Combining concepts
- Tuesday:
  - Exam exercises

1. Variables, Expressions and Statements
2. Functions
3. Conditionals
4. While Loop
5. For Loop
6. Lists
7. Dictionaries
8. Exceptions
9. Files
10. Tests
11. Tuples*
12. Classes*
13. Class methods
14. Inheritance*
15. Comprehension

Exception Handling

- AttributeError, IndexError, KeyError
- NameError
- TypeError
- ValueError
- IOError
- ZeroDivisionError
- ...

Practice by trying to get all!

```
age_str = input("Enter your age")
try:
    age = int(age_str)
except ValueError:
    print("Could not parse input as number")
```

```
age_str = input("Enter your age")
try:
    age = int(age_str)
except ValueError as e:
    print("Could not parse input as number")
    print(e)
    print(e.args)
```

Dictionaries

Mappings of keys to values:

```
person = {
    "first_name": "John",
    "last_name": "Doe",
    "nationality": "Canada",
    "birth_year": 1980
}
```

Accessing entries:

```
person["first_name"] # "John"
```

Iterating over dictionaries:

```
for entry in person:
    print(entry)
```

This will yield the keys:
"first_name", "last_name", "nationality", "birth_year"

Iterating over key-value-pairs:

```
for key, value in person.items():
    print(f'{key}, {value}')
```

Operations on dictionaries:

```
d = {0: 'zero', 1: 'one', 2: 'two'}

d[2]
d[2] = 'TWO'
d[3] # KeyError
d.get(3) # None
d.setdefault(2, 'n')
d.setdefault(3, 'n')

d.keys()
d.items()

d1.update(d2)
```

To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary

Print the name of child 2:

```
print(myfamily["child2"]["name"]) # Tobias
```

You can loop through a dictionary by using the items() method like this
Loop through the keys and values of all nested dictionaries:

```
for x, obj in myfamily.items():
  print(x)

  for y in obj:
    print(y + ':', obj[y])
```

```
child1
name: Emil
year: 2004
child2
name: Tobias
year: 2007
child3
name: Linus
year: 2011
```

Nested dictionaries

file = a sequence of bytes on a storage device
Many file formats are a sequence of text characters - e.g. the formats .txt, .html, .csv or .py.
The content of text files can be represented as strings, other file contents can be represented as byte sequences.

Writing a text file

```
file = open("message.txt", "w", encoding="utf-8")
file.write("hello world\n")
file.write("end\n")
file.close()
```

Reading a text file (r = standard)

```
file = open("message.txt", encoding="utf-8")
content = file.read()
file.close()
print(content)
```

File modes (combinable)

t: text mode (default)
b: binary

r: reading (default)
w: (over)writing
a: appending

File I/O

```
with open("todos.txt", encoding="utf-8") as file_obj:
    content = file_obj.read()
```

The file will be closed automatically when the program leaves the indented block.

```
with open("myfile.txt", encoding="utf-8") as file:
    for line in file:
        print(line)
```

Methods / Attributes:

- .close()
- .mode
- .read() (read the entire file)
- .read(10) (read the next 10 bytes)
- .readline() (read the next line)

File I/O

Entries are separated by commas, **usually** surrounded by round brackets.

```
empty_tuple = ()
single_value = ('Thomas', )
single_value = 'Thomas',
two_values = ('Thomas', 'Smith')
two_values = 'Thomas', 'Smith'
```

Unpacking tuples:

```
time = (23, 45, 0)
hour, minute, second = time
```

Swapping variables:

```
a, b = b, a
```

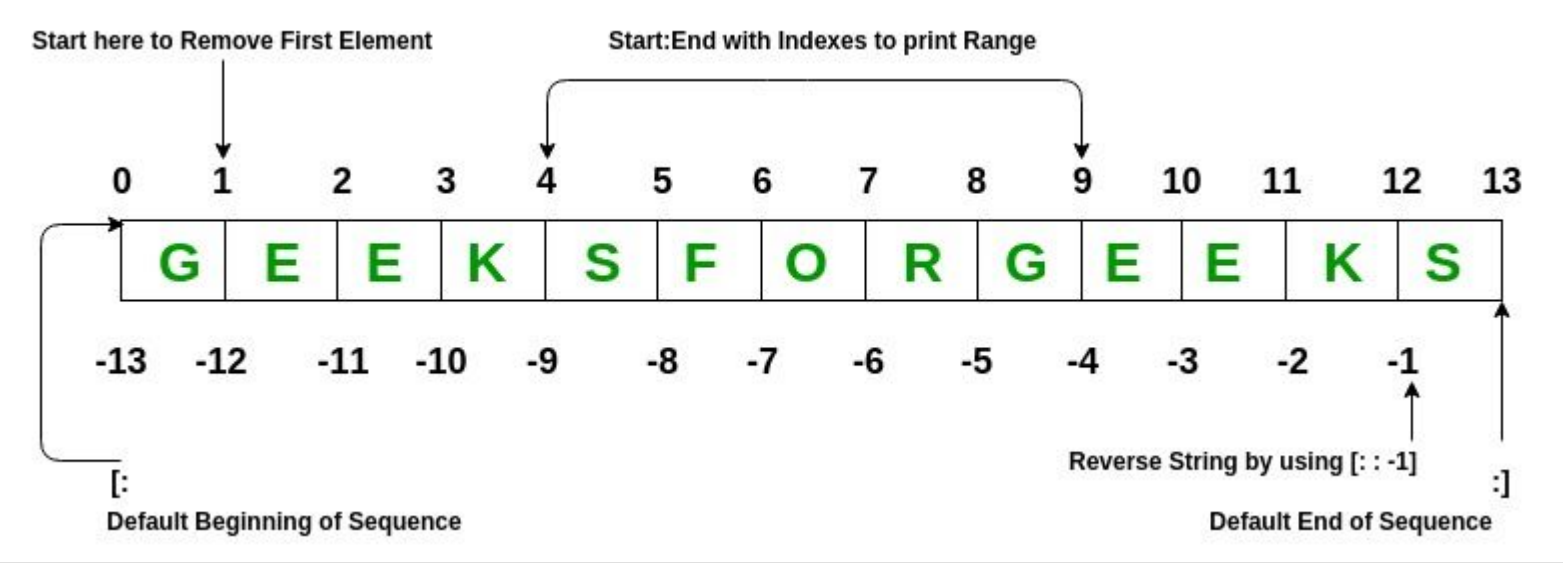
Tuples

Tuple items are **ordered**, **unchangeable**, and a**llow duplicate values**.

Tuple items are indexed, the first item has index [0], the second item has index [1] ...

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Dictionary** is a collection which is ordered and changeable. No duplicate members.



Tuples - slicing

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Tuple methods

- Loops but in the form of a sentence
- Read backwards!

```
squares = []
for x in range(5):
    squares.append(x**2)
```

=

```
squares = [x**2 for x in range(5)]
```

List comprehensions enable the creation of lists based on existing lists

Transforming:

```
names = ["Alice", "Bob", "Charlie"]
uppercase_names = [name.upper() for name in names]
```

Filtering:

```
amounts = [10, -7, 8, 19, -2]
positive_amounts = [amount for amount in amounts if amount > 0]
```

General syntax:

new_list = [new_entry **for** entry **in** old_list]

new_list = [new_entry for entry **in** old_list **if** condition]

Comprehension

```
colors = {
  'red': '#ff0000',
  'green': '#008000',
  'blue': '#0000ff',
}
m_colors = { color: colors[color][1:] for color in colors}
```

Or

```
m_colors = {
    name: value[1:] for name, value in colors.items()
}
```

Dictionary comprehensions

Classes may represent various things, e.g.:

- a Message inside an e-mail program
- a user of a website
- a car in a racing game
- a shopping basket in an online shop
- a bank account
- a data set to be analysed

The **definition of a class** usually encompasses:

- a "data structure" (attributes)
- a "behavior" (methods)

Create an **object** named p1, and print the value of **x**:

```
p1 = MyClass()
print(p1.x)
```

example: class TextIOWrapper can represent a text file (is created when calling open())

attributes:

- closed
- encoding
- mode (e.g. r=read, w=write)
- name (filename)
- ...

methods:

- close()
- read()
- write()
- ...

OOP (Classes & Inheritance)

```python
class MyClass():

    # the method __init__ initializes the object
    def __init__(self):
        # inside any method, self will refer
        # to the current instance of the class
        self.message = "hello"

instance = MyClass()
instance.message # "hello"
```

The __init__() Function:
To understand the meaning of classes we have to understand the built-in __init__() function.
All classes have a function called __init__(), which is always executed when the class is being initiated.
Use the __init__() function to **assign values to object properties**, or other operations that are necessary to do when the object is being created:

Create a class named Person, use the __init__() function to assign values for name and age:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)
print(p1.name)
print(p1.age)
```

The **__str__() function** controls what should be returned when the class object is represented as a string.
If the __str__() function is not set, the string representation of the object is returned:

The string representation of an object **WITHOUT** the __str__() function:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1)
```

```
# <__main__.Person object at 0x15039e602100>
```

The string representation of an object **WITH** the __str__() function:

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age
  def __str__(self):
    return f"{self.name}({self.age})"

p1 = Person("John", 36)
print(p1)
```

```
# John(36)
```

Objects can also contain methods. Methods in objects are functions that belong to the object.

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self, ..., ..., ....):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

The self parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

It **does not** have to be named self, you can call it whatever you like, but it has to be the **first** parameter of any function in the class

often we can use some class(es) as the **basis** for another class

e.g.:
User class as the basis of the AdminUser class
TicTacToeGame as the basis of TicTacToeGameGUI

inheritance:
an AdminUser **is** a User

```
class User():
    ...
class AdminUser(User):
    ...
```

the AdminUser class automatically inherits all existing methods from the User class

composition:
TicTacToeGameGUI could use TicTacToeGame in the background

```
class TicTacToeGame():
    ...
class TicTacToeGameGUI():
    def __init__(self):
        self.game = TicTacToeGame()
```

Inheritance and composition

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.
Child class is the class that inherits from another class, also called derived class.

```
class Student(Person):
  pass
```

Use the **pass** keyword when you do not want to add any other properties or methods to the class.

```
class Student(Person):
  def __init__(self, fname, lname):
    #add properties etc.
```

When you add the **__init__() function**, the child class will **no longer inherit** the parent's __init__() function.

Inheritance - **__init__()**

Python also has a **super()** function that will make the child class inherit all the methods and properties from its parent:

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
```

By using the super() function, you do not have to use the name of the parent element, it will **automatically inherit the methods and properties** from its parent.

If you add a method in the child class with **the same name** as a function in the parent class, the inheritance of the parent method will be **overridden**.

Inheritance - super()

```python
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name  # Initialize the name attribute

    def speak(self):
        pass  # Placeholder method to be overridden by child classes

# Child class inheriting from Animal
class Dog(Animal):
    def speak(self):
        return f"{self.name} barks!"  # Override the speak method

# Creating an instance of Dog
dog = Dog("Buddy")
print(dog.speak())  # Output: Buddy barks!
```

Inheritance - overriding

Study plan & resources

- W3Schools Python Tutorial
  - Self-check quiz
- GeeksforGeeks
  - Per-topic quizzes
- Slides & Exercises
- Cheatsheet(s)

- Advanced:
  - Codewars
  - Leetcode

- Friday:
  - Go over basics
- Saturday:
  - Dictionaries
  - Nesting
- Sunday:
  - OOP
- Monday:
  - Combining concepts
- Tuesday:
  - Exam exercises

- Write a helper function that removes special characters and extra spaces from a string.
- Sort words in a given sentence alphabetically.
- Create a list comprehension that extracts even numbers from a list of numbers.
- Given a tuple of student names, write a function to return a sorted tuple without duplicates.

Friday

- Create a Person class with attributes name and age. Then, make a Student subclass that adds student_id.
- Implement a class PasswordChecker that checks password strength based on rules (e.g., length, special characters).
- Write a class Song that stores title and artist, and a method to clean up song titles.
- Modify the Song class so that a Playlist class can store multiple songs and sort them alphabetically.

Saturday

- Given a dictionary of students {“Alice”: 85, “Bob”: 90, “Charlie”: 78}, write a function to return the student with the highest score.
- Modify this dictionary so that each student has a dictionary of {"Math": 85, "English": 90} and find the highest scorer in Math.
- Write a program that reads a CSV file of student scores and converts it into a dictionary.
- Implement a class that loads student data from a JSON file and provides methods to retrieve info.

Sunday

- Build a User class with a method to save user data to a file and another method to retrieve it.
- Extend the password checker by adding a dictionary of banned passwords.
- Create a function that reads a text file of song lyrics, removes special characters, and counts word occurrences using a dictionary.

Monday

- Given a list of tuples (name, age), use list comprehensions to extract names of people over 18.
- Create a BankAccount class that handles deposits and withdrawals, with error handling for insufficient funds.
- Given a deeply nested dictionary, write a function to extract a specific value by providing a list of keys.
- Create a program that takes a user’s full name as input, cleans extra spaces, and formats it as "Last, First".

Tuesday

- The name of the function matches the one in the description 0.5
- The function has a doc-string. 0.5
- The function has type-hints. 0.5
- The function has correct parameters. 0.5
- The function performs a search (can be incorrect) 3
- T1 The function returns the correct artist 3
- T2 The function returns the correct “not found” message if no artist was found (if search is incorrect and function does not always return “not found”, award the 2 points) 2

On Grading

- print(“is your best friend”)
- *Learn to read* error messages
- Make a *battle plan* before you start coding (break it down into parts)
- Get *fast* & make mistakes
- **LBYL**: Look before you leap
- **EAFP**: It's easier to ask for forgiveness than permission

General tips!

There are
NOT official
materials

Disclaimer!

STORM MENTOR

2024 - 2025

Thank You !